

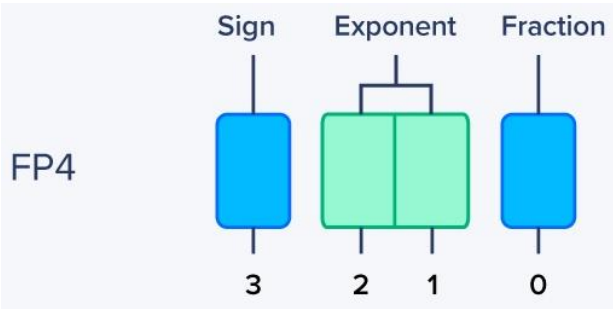
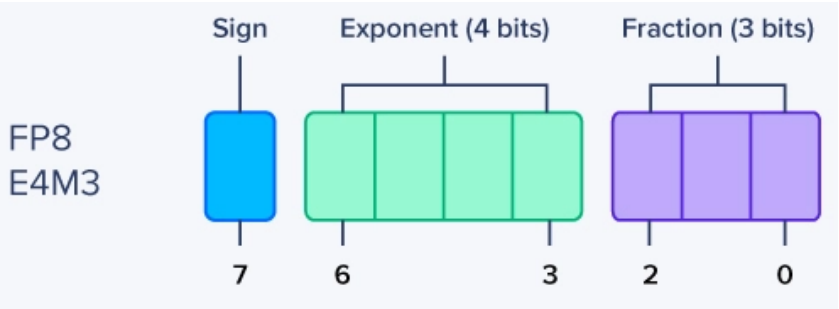


Lab07 Code Review

張皓庭

Floating Point Review

bf16



Format	Bias Exponent	X_{format}	X_{real}
FP4	1	0 11 1	$(-1)^0 * 2^{(3-1)} * (1.1)$
FP8	7	0 1101 010	$(-1)^0 * 2^{(13-7)} * (1.010)$
bf16	127	1 10000111 0011010	$(-1)^1 * 2^{(135-127)} * (1.0011010)$

參考資料:
<https://medium.com/@manyi.yim/bfloat16-vs-float32-vs-float16-back-to-the-basics-80d4aec49ca8>
<https://www.exxactcorp.com/blog/hpc/what-is-fp8-fp6-fp4>

Lab

- 每組測資 (共有 8 個 data) 共用一個 **FP8 scaling factor**
- 每組測資 (共有 8 個 data) 共用一個 **bfloat16 scaling factor**
- 為了方便計算，本次 bfloat16 scaling factor 直接設定為 0.5_{decimal} ，也就是以下的 **$bf16\ sf = 0.5_{\text{decimal}}$**



out_data (本次 Lab 的 output)

$$= \sum_{i=0}^3 (real\ a_i * real\ b_i)$$

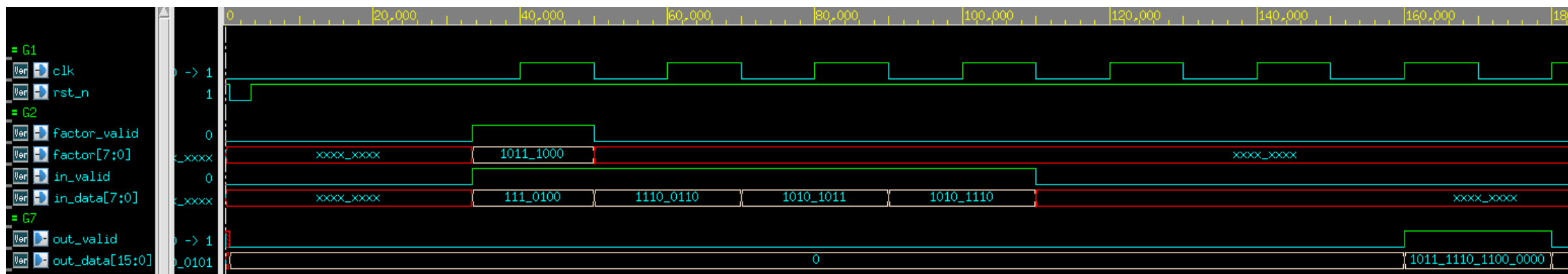
$$= \sum_{i=0}^3 ((FP4\ a_i * FP8\ sf_{a1} * bf16\ sf) * (FP4\ b_i * FP8\ sf_{b1} * bf16\ sf))$$

$$= \sum_{i=0}^3 (FP8\ sf_1 * (bf16\ sf)^2 * FP4\ a_i * FP4\ b_i)$$

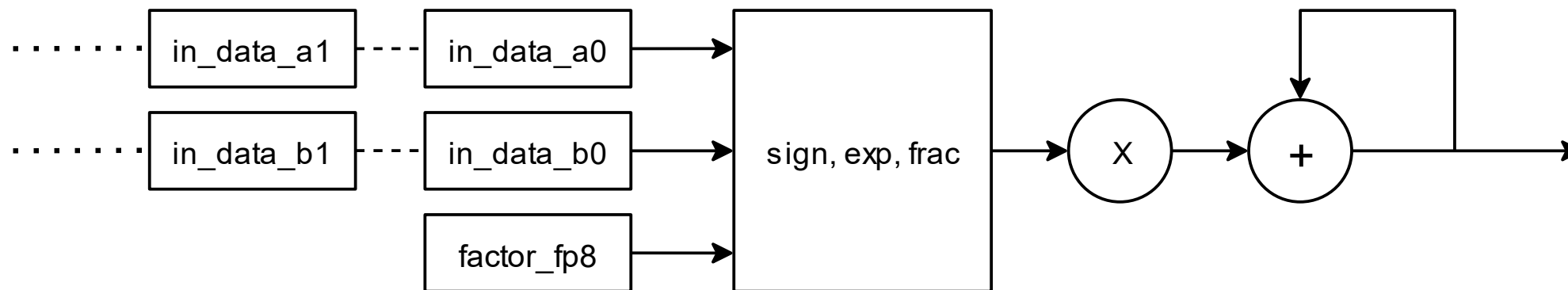
$$= FP8\ sf_1 * (bf16\ sf)^2 * \sum_{i=0}^3 (FP4\ a_i * FP4\ b_i)$$

Waveform

- BF16 (E8M7) scaling factor 固定為 0.5_{decimal}
- FP8 (E4M3) scaling factor (sign, exp, frac): 10011000
- 4 in_data (sign, exp, frac)
 - {a0, b0} = {0111, 0100}
 - {a1, b1} = {1110, 0110}
 - {a2, b2} = {1010, 1011}
 - {a3, b3} = {1010, 1110}



Block Diagram



- $\text{factor}_{\text{bf16}} = 0.5$, 按照公式, 可以直接從 $\text{exponent} - 2$ 即可
- method
 1. 先完成 $(a_{\text{fp4}} * b_{\text{fp4}})$ 的累加, 最後再乘上 $\text{factor}_{\text{fp8}}$
 2. a_{fp4} 和 b_{fp4} 都先變成 bf16 格式再做乘加

Design Logic (x)

- 先把 sign, exponent, fraction 讀出來
- 直接做 $\text{fraction}_a * \text{fraction}_b$
- exponent 相加, 並對齊 bf16 的 bias

```
144 assign sign_a = data_a[3];
145 assign sign_b = data_b[3];
146 assign exp_a  = data_a[2:1] - 1;
147 assign exp_b  = data_b[2:1] - 1;
148 assign frac_a = {1'b1, data_a[0], 6'b0};
149 assign frac_b = {1'b1, data_b[0], 6'b0};
150 assign sf_sign = sf[7];
151 assign sf_exp  = sf[6:3] + 8'd120;
152 assign sf_frac = { sf[2:0], 4'b0 };
```

```
201 assign sign_product_next = sign_a ^ sign_b;
202 assign frac_product_temp = frac_a * frac_b;
203 assign exp_product_temp = exp_a + exp_b;
204
205 always_comb begin
206     // Normalization
207     if (frac_product_temp[15]) begin
208         frac_product_next = frac_product_temp[14:8];
209         exp_product_next = exp_product_temp + 1 + 127;
210     end
211     else begin
212         frac_product_next = frac_product_temp[13:7];
213         exp_product_next = exp_product_temp + 127;
214     end
215 end
```

Design Logic (+)

- 判斷正負號
- 判斷 exponent 大小
 - 相同: 可以直接相加減
 - 不同: 先 shift 再相加減
- 加減法
 - 加法: 判斷有無進位, exponent 不變或是 +1
 - 減法: 利用 leading one, 找出相加之後需要 shift 多少

```

291  if (frac_sum_temp[7])      sh = 3'd0;
292  else if (frac_sum_temp[6]) sh = 3'd1;
293  else if (frac_sum_temp[5]) sh = 3'd2;
294  else if (frac_sum_temp[4]) sh = 3'd3;
295  else if (frac_sum_temp[3]) sh = 3'd4;
296  else if (frac_sum_temp[2]) sh = 3'd5;
297  else if (frac_sum_temp[1]) sh = 3'd6;
298  else if (frac_sum_temp[0]) sh = 3'd7;
299  else
      sh = 3'd0;

```

```

// same sign
if (sum[15] == product[15]) begin
    // exponent equal
    if (sum[14:7] == product[14:7]) begin...
    end

    // exponent not equal
    else if (sum[14:7] > product[14:7]) begin...
    end

    else begin...
    end
end
// different sign
else begin
    // exponent equal
    if (sum[14:7] == product[14:7]) begin...
    end

    // exponent not equal
    else if (sum[14:7] > product[14:7]) begin...
    end

    else begin...
    end
end

```